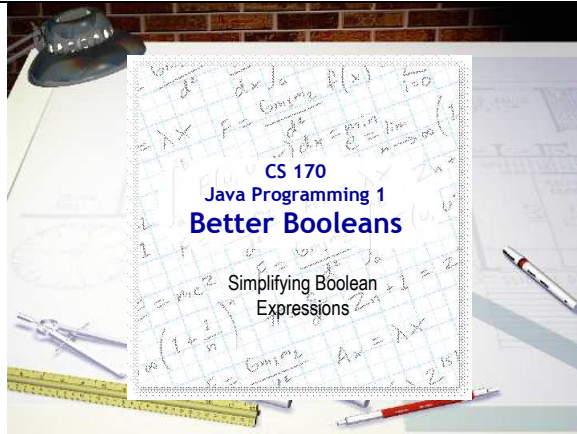




Slide 1

CS 170

Java Programming 1

Better Booleans

Duration: 00:01:15
Advance mode: Auto

Notes:

Hello. Welcome to the CS 170 Java Programming 1 lecture on Better Booleans.

This week you were introduced to Java's Boolean type and you learned how to put it to work writing predicate methods and making selection statements. You also learned how to produce Boolean values using the relational operators and how to combine Boolean expressions using the logical operators.

While the Boolean type is simple, with only two values, as you've found out by now, combined Boolean expressions can end up becoming very complex. Since Boolean expressions are the cornerstone of computer decision making and repetition, it's important to learn how to be sure that your Boolean expressions are correct.

In this lecture we'll look at some common pitfalls, and I'll give you some hints about how can simplify those expressions that become incomprehensible.

As you begin to write code that uses AND and OR, you're almost certain to occasionally confuse the two. When you do, you'll find yourself writing two kinds of erroneous condition statements: the impossible condition and the unavoidable condition.

Let's take a look at both.

- When no combination of operands could ever be true
 - "Give a 25% discount to children who are under 13 and to patrons who are 65 and older"
- A "literal" reading results in code like this:


```
if ( age < 13 && age > 65 )
    discount = .25;
else
    discount = 0;
```
- No possible value for age where this could be true

Slide 2

The Impossible Condition

Duration: 00:01:11
Advance mode: Auto

Notes:

An impossible condition occurs when no conceivable combination of values could ever make the condition true. Let's suppose you take a job automating the box office at your local cinema chain. As your first task, the boss asks you to implement some code that:

Give a 25% discount to children who are 12 and under and to patrons who are 65 or older.

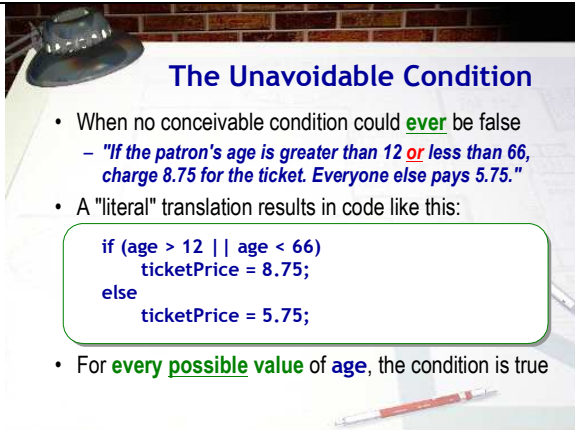
If you're in a hurry, you may notice the AND in the problem statement, and write your code like this:

```
if (age <= 12 && age >= 65)
    discount = .25;
else
    discount = 0.0;
```

This is an impossible condition because for every possible value that the variable age could take on, the expression still evaluates to false. If the value in the variable age is less than 12, then, by definition, it cannot be greater than 65. If age contains a value greater than 65, then by definition, it cannot

be less than 12.

Because AND requires that both sides be true before the entire expression is true, this condition can never be true. It is impossible.



The Unavoidable Condition

- When no conceivable condition could **ever** be false
 - "If the patron's age is greater than 12 **or** less than 66, charge 8.75 for the ticket. Everyone else pays 5.75."
- A "literal" translation results in code like this:

```
if (age > 12 || age < 66)
    ticketPrice = 8.75;
else
    ticketPrice = 5.75;
```
- For **every possible value** of **age**, the condition is true

Slide 3

The Unavoidable Condition

Duration: 00:01:37

Advance mode: Auto

Notes:

The impossible condition occurs when the expression, by definition, could never be true; the unavoidable condition occurs in the opposite case. An unavoidable condition occurs when no conceivable combination of values used as operands could ever make the condition false.

Let's suppose that you've avoided getting fired and that your boss gives you a second chance on the ticket project. After all, maybe he thinks that the problem is with the percentages since they weren't listed as part of your algebra student-learning outcomes. So, he figures out the percentage and just tells you the ticket price like this:

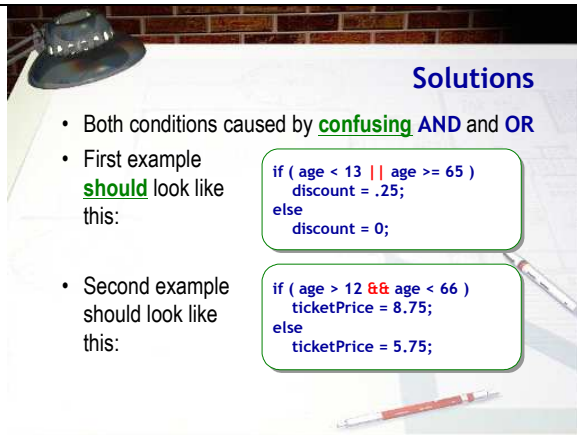
If the patron's age is greater than 12 or less than 65 we'll charge them 8.75 for their ticket. All others will pay the discounted price of 5.75.

This time you're much more careful. You painstakingly translate the specification, checking each word. When you've translated it into Java code you end up with:

```
if (age > 12 || age < 65)
    ticketPrice = 8.75;
else
    ticketPrice = 5.75;
```

The only problem? for every value of age, the entire expression is true. If age is less than 12, it fails the first test, but passes the second. If age is greater than 65, it passes the first test, while failing the second. If age falls between 12 and 65 it passes both tests.

For every possible value of age, the expression is true. Everyone pays full price because the condition is unavoidable.



Solutions

- Both conditions caused by **confusing AND and OR**
- First example **should** look like this:


```
if ( age < 13 || age >= 65 )
    discount = .25;
else
    discount = 0;
```
- Second example should look like this:


```
if ( age > 12 && age < 66 )
    ticketPrice = 8.75;
else
    ticketPrice = 5.75;
```

Slide 4

Solutions

Duration: 00:00:43

Advance mode: Auto

Notes:

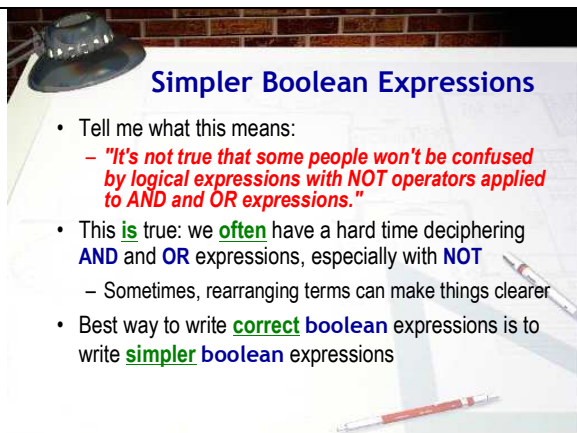
Both of these problems occur because when we use AND and OR in informal speech, we determine which one is actually meant by the context. A computer language, though, can't do that; you have to explicitly say what you mean, following the formal rules of logic.

The solution to both of these problems is to double-check each time you use AND and OR, to make sure you are using the correct one. Our first example can be fixed by using OR instead of AND, like this:

```
if (age < 13 || age >= 65)
    discount = .25;
else
    discount = 0.0;
```

In the same way, the unavoidable condition can be fixed by replacing OR with AND, like this:

```
if (age > 12 && age < 66)
    ticketPrice = 8.75;
else
    ticketPrice = 5.75;
```



Simpler Boolean Expressions

- Tell me what this means:
 - "It's not true that some people won't be confused by logical expressions with NOT operators applied to AND and OR expressions."*
- This **is** true: we **often** have a hard time deciphering **AND** and **OR** expressions, especially with **NOT**
 - Sometimes, rearranging terms can make things clearer
- Best way to write **correct boolean** expressions is to write **simpler boolean** expressions

Slide 5

Simpler Boolean Expressions

Duration: 00:01:09

Advance mode: Auto

Notes:

Can you tell me what this sentence means? It's not true that some people won't be confused by logical expressions with NOT operators applied to AND and OR expressions.

If you're like me (and most everyone else), you're initial reaction is: Huh?

Well, let's see if I can rephrase the statement:

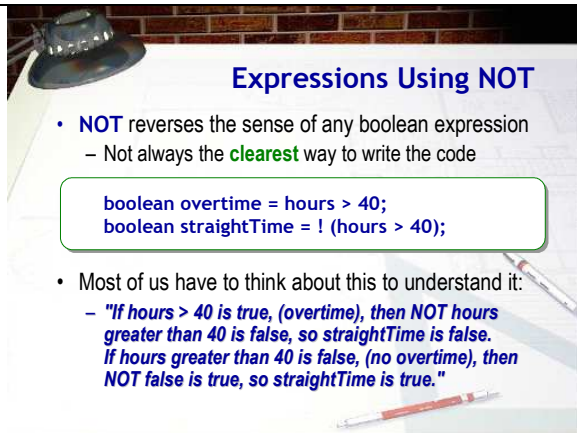
It is true; you and I often have a hard time deciphering AND and OR expressions, especially when combined with the NOT operator.

That's better, and easier to understand, isn't it? Sometimes, just rearranging the terms in a condition can make it easier to understand. Similarly, the best way to write correct Boolean expressions, is to write simpler Boolean expressions.

There are two "tricks" that can help you here:

1. use alternatives to the NOT operator in relational expressions
2. use De Morgan's laws to simplify compound expressions using NOT

Let's look at alternatives to NOT first.



Expressions Using NOT

- **NOT** reverses the sense of any boolean expression
 - Not always the **clearest** way to write the code

```
boolean overtime = hours > 40;
boolean straightTime = !(hours > 40);
```

- Most of us have to think about this to understand it:
 - "If *hours > 40* is true, (overtime), then **NOT** *hours greater than 40* is false, so *straightTime* is false. If *hours greater than 40* is false, (no overtime), then **NOT** false is true, so *straightTime* is true."

Slide 6

Expressions Using NOT

Duration: 00:00:51

Advance mode: Auto

Notes:

You can use the NOT operator to reverse the sense of any Boolean expression. Often, though, that's not the clearest way to write your code. In a payroll processing program, for instance, you may have to check for overtime hours using an expression like this:

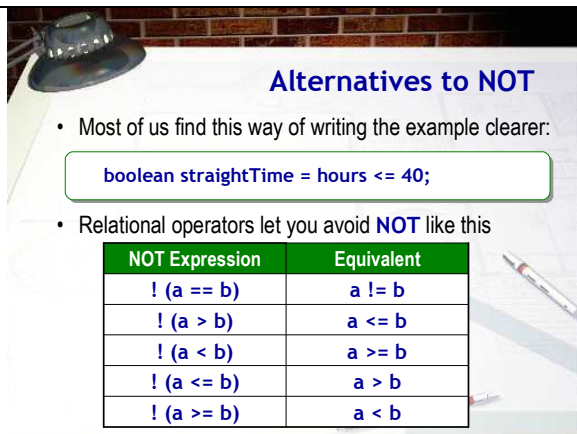
```
boolean overtime = hours > 40;
```

You can use the NOT operator to reverse an expression like this:

```
boolean straightTime = !(hours > 40);
```

Most of us, however, don't find that very clear; we have to stop and think about it:

"Hmmm. If hours greater than 40 is true, then NOT hours greater than 40 is false, so straightTime is false. If hours greater than 40 is false, (no overtime), then NOT false is true, so straightTime is true."



Alternatives to NOT

- Most of us find this way of writing the example clearer:


```
boolean straightTime = hours <= 40;
```
- Relational operators let you avoid **NOT** like this

NOT Expression	Equivalent
! (a == b)	a != b
! (a > b)	a <= b
! (a < b)	a >= b
! (a <= b)	a > b
! (a >= b)	a < b

Slide 7

Alternatives to NOT

Duration: 00:00:59

Advance mode: Auto

Notes:

That's too much work; most of us would find the following much clearer:

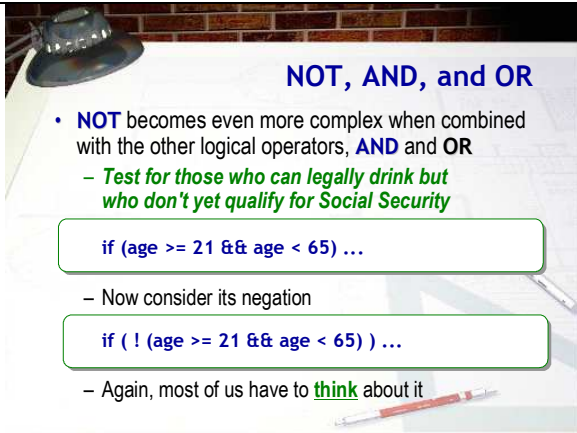
```
boolean straightTime = hours <= 40;
```

Notice that the only difference is that we've replaced the combination of NOT and a relational operator with a different relational operation: one that expresses the relationship in a more straightforward way.

When working with the relational operators, you can often avoid using NOT, by using a different relational operator like this.

Instead of writing NOT a == b, you can write a != b. Instead of writing ! (a > b) you can write a <= b, and so on.

Anytime you find yourself using ! with a relational expression, try to "rephrase" the expression using one of these equivalents.



NOT, AND, and OR

- **NOT** becomes even more complex when combined with the other logical operators, **AND** and **OR**
 - *Test for those who can legally drink but who don't yet qualify for Social Security*

```
if (age >= 21 && age < 65) ...
```

- Now consider its negation

```
if ( ! (age >= 21 && age < 65) ) ...
```

- Again, most of us have to **think** about it

Slide 8 
NOT, AND, and OR
 Duration: 00:00:58
 Advance mode: Auto

Notes:

When combined with the AND and OR operators, NOT quickly becomes even more complicated and confusing. The following code checks for folks who can drink (in California), but who don't qualify for Social Security:

```
if (age >= 21 && age < 65) . . .
```

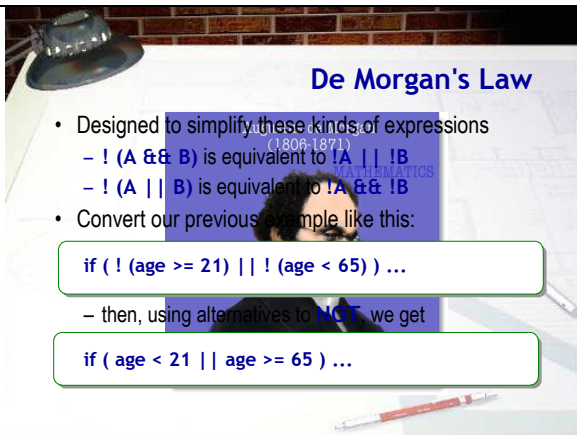
None of you should have a problem with that; it's really clear and straightforward.

But, how about its negation?

```
if ( ! (age >= 21 && age < 65) ) . . .
```

Without the first piece of code to act as a guide, most of us would have to carefully think to figure out what was really being tested. And code that you have to carefully think about to understand is code that can easily hide errors.

So, how can we make this piece of code clearer?



De Morgan's Law

- Designed to simplify these kinds of expressions
 - $\neg (A \ \&\& \ B)$ is equivalent to $\neg A \ || \ \neg B$
 - $\neg (A \ || \ B)$ is equivalent to $\neg A \ \&\& \ \neg B$
- Convert our previous example like this:

```
if ( ! (age >= 21) || ! (age < 65) ) ...
```

- then, using alternatives to **NOT** we get

```
if ( age < 21 || age >= 65 ) ...
```

Slide 9 
De Morgan's Law
 Duration: 00:01:28
 Advance mode: Auto

Notes:

These are the kinds of expressions that De Morgan's law, (named after Augustus de Morgan, a 19th Century logician, and tutor to Lady Ada Lovelace), helps to simplify.

There are two versions of the law: one for AND expressions, and one for OR expressions.

$\neg (A \ \&\& \ B)$ is equivalent to $\neg A \ || \ \neg B$

$\neg (A \ || \ B)$ is equivalent to $\neg A \ \&\& \ \neg B$

Notice that in both cases:

- first the NOT is distributed to both of the Boolean sub-expressions.
- then, the && or || connection is changed to the opposite: AND to OR and OR to AND.

Using this, we can convert our expression to:

```
if ( !(age >= 21) || !(age < 65) ) . . .
```

which certainly isn't much, (if any), improvement.

BUT, as they say in the TV ads, That's Not All! Now we can use the alternatives for the NOT operator that we saw in the previous section to replace each of the sub-expressions like this:

```
if ( age < 21 || age >= 65 ) . . .
```

which *is* much clearer. (Read the condition out loud to yourself, to see the difference.)

Express Yourself

- **Exercise 1:** Simplify these Boolean expressions

- $\neg (x > 0 \ \&\& \ y > 0)$
- $\neg (x \neq 0 \ || \ y \neq 0)$
- $\neg (\text{country.equals("US")} \ \&\& \ \neg \text{state.equals("HI")} \ \&\& \ \neg \text{state.equals("AK")})$
- $\neg (x \% 4 \neq 0 \ || \ \neg (x \% 100 == 0 \ \&\& \ x \% 400 == 0))$

Slide 10

Express Yourself

Duration: 00:00:29

Advance mode: Auto

Notes:

Let's finish up this week by simplifying some Boolean expressions. Create a new section in your lecture-lab exercise document and reduce these expressions to their simplest terms using De Morgan's law and the relational alternatives to NOT.

As you do your work, please type in the sub-expressions as you go. That is, for each problem, you'll have two or more intermediate steps.